UATM GASA FORMATION

Université Africaine de Technologie et de Management

RAPPORT DE PROJET

Conception et développement d'une application de gestion de stock

Application StockPro

Filière : Système Informatique et Logiciel (SIL3)

Année académique : 2026 – 2027

Fait par : KOUANDZI MIETTE JUNIOR

Sous la supervision de Madame : Falonne Vanessa ZOMAHOUN

SOMMAIRE

Introduction générale.....3
1. Fonctionnalités de l'application.....3
2. Analyse.....4
2.1. Diagramme de cas d'utilisation.....4
2.2. Description des acteurs.....4
2.3. Relations entre les cas d'utilisation.....4
3. Conception.....5
3.1. Diagramme de classes.....5
3.2. Description des classes.....5
3.3. Relations et règles de gestion.....6
4. Modèle logique de données relationnel.....7
4.1. Règles de passage.....7
4.2. Schéma relationnel.....7
5. Implémentation de la base de données.....7
6. Réalisation de l'application.....8
6.1. Environnement de développement.....8
6.2. Interfaces de l'application.....8
7. Extraits de code.....11
Conclusion générale.....11
Annexe A. Script SQL complet.....12

Introduction générale

La gestion des stocks consiste à suivre les produits disponibles et leurs mouvements. Un suivi manuel entraîne des erreurs de quantité et rend difficile le repérage des produits presque épuisés. Une application dédiée centralise ces informations, enregistre les entrées et les sorties, calcule les quantités disponibles et signale les stocks faibles.

StockPro répond à ce besoin : elle gère les produits et leurs catégories, enregistre les mouvements de stock et génère des alertes lorsque le stock devient insuffisant.

Problématique : comment concevoir une application qui assure un suivi fiable des stocks, gère les mouvements des produits et prévient les situations de stock faible ?

Hypothèses :

- H1 : une application informatisée améliore le suivi des stocks.
- H2 : l'enregistrement des entrées et des sorties donne des quantités plus précises.
- H3 : un système d'alerte permet de repérer rapidement les stocks faibles.

Choix du sujet : le suivi des stocks est un besoin concret et mobilise l'analyse, la modélisation UML, les bases de données et le développement web.

1. Fonctionnalités de l'application

Tableau 1 : Fonctionnalités de StockPro

Fonctionnalité	Description
Authentification	Connexion obligatoire, accès selon le rôle.
Gestion des produits	Ajouter, modifier, supprimer, consulter. Un produit appartient à une catégorie.
Gestion des catégories	Créer et organiser les catégories.
Gestion des fournisseurs	Enregistrer et mettre à jour les fournisseurs.
Entrées de stock	Enregistrer une arrivée ; la quantité augmente automatiquement.
Sorties de stock	Enregistrer un retrait ; la quantité diminue automatiquement.
État du stock	Consulter les quantités disponibles.
Alertes de stock faible	Signaler les produits ayant atteint leur seuil.
Inventaire	Comparer quantités enregistrées et quantités réelles.
Statistiques globales	Suivre le stock avec des indicateurs synthétiques.
Journal des activités	Garder une trace des actions réalisées.
Gestion des accès	Gérer les comptes utilisateurs et leurs droits.

2. Analyse

2.1. Diagramme de cas d’utilisation

Le diagramme de cas d’utilisation montre ce que chaque acteur peut faire avec le système.

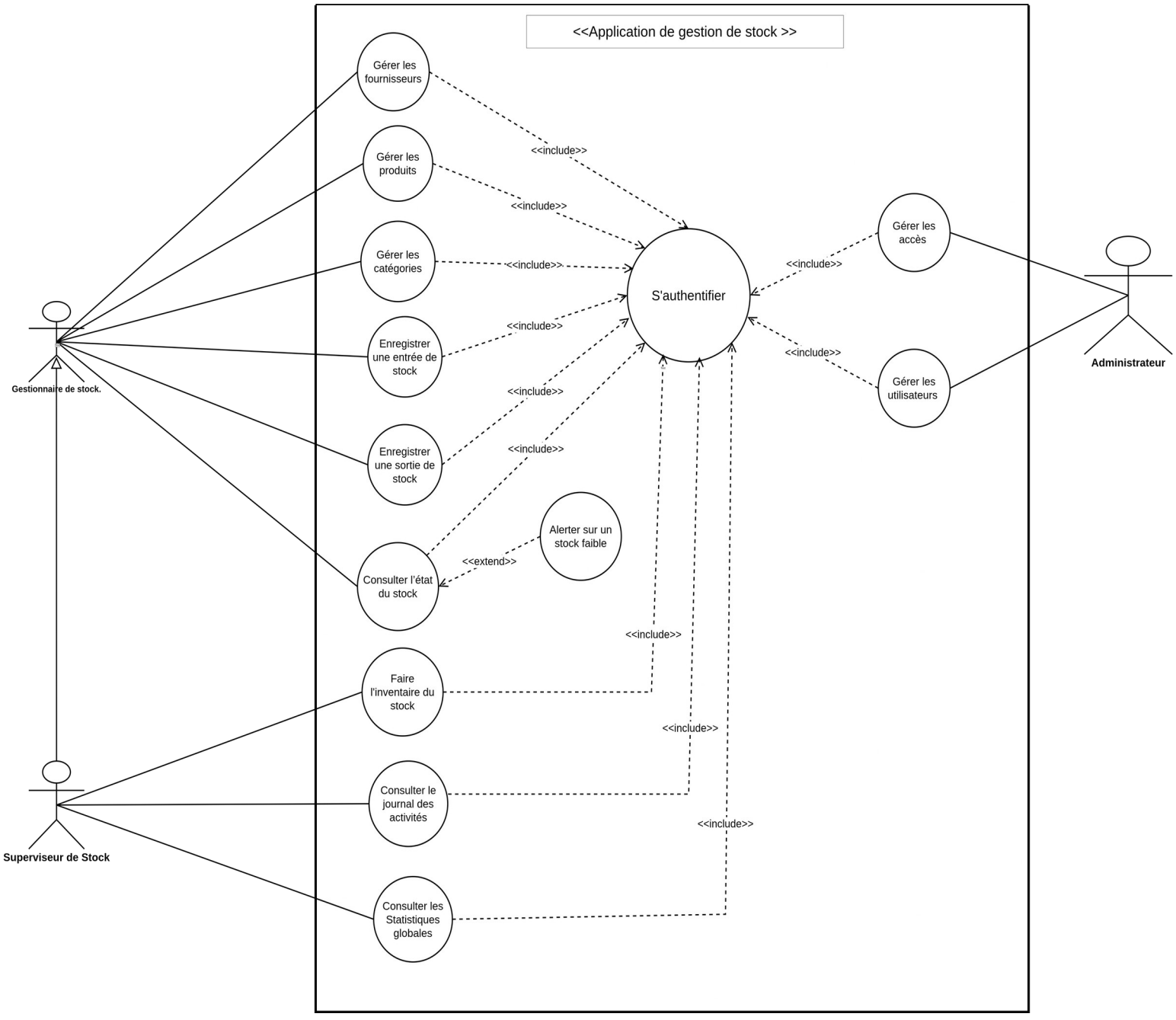


Figure 1 : Diagramme de cas d’utilisation

2.2. Description des acteurs

Tableau 2 : Acteurs du système

Acteur	Rôle	Cas d'utilisation
Gestionnaire de stock	Gère le stock au quotidien.	Gérer produits, catégories, fournisseurs Enregistrer une entrée ou une sortie Consulter l'état du stock
Superviseur de stock	Assure le suivi global.	Faire l'inventaire Consulter le journal des activités Consulter les statistiques
Administrateur	Gère les comptes et les droits.	Gérer les accès Gérer les utilisateurs

Le Superviseur hérite du Gestionnaire : il dispose de toutes ses fonctionnalités en plus des siennes.

2.3. Relations entre les cas d'utilisation

- **include** : tous les cas d'utilisation incluent « S'authentifier ».
- **extend** : « Alerter sur un stock faible » étend « Consulter l'état du stock » ; l'alerte apparaît seulement quand un produit atteint son seuil.

3. Conception

3.1. Diagramme de classes

Le diagramme de classes représente les classes, leurs attributs, leurs méthodes et leurs relations.

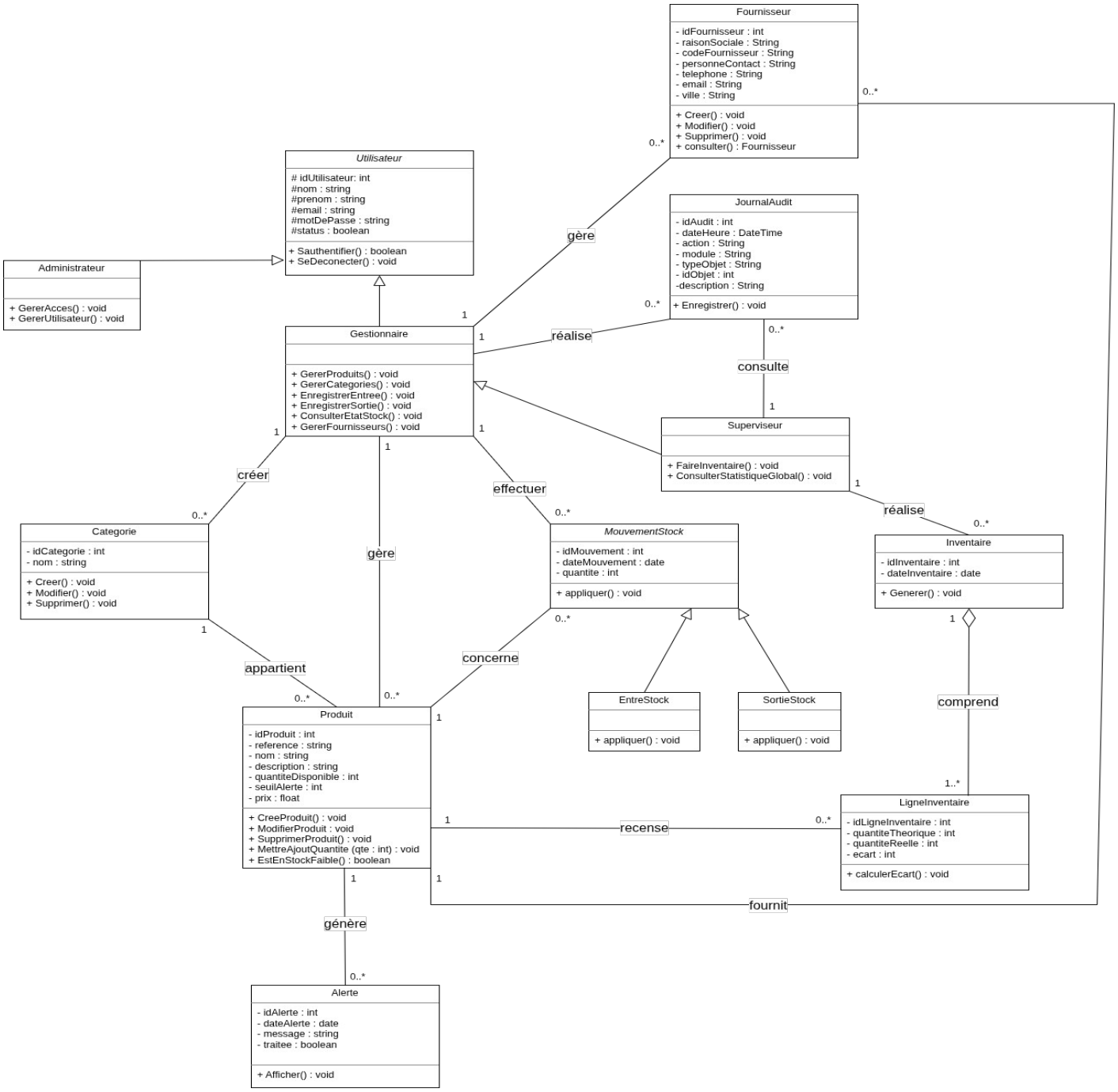


Figure 2 : Diagramme de classes

3.2. Description des classes

Tableau 3 : Description des classes

Classe	Attributs principaux	Rôle
Utilisateur (abstraite)	idUtilisateur, nom, prenom, email, motDePasse, status	Représente tout utilisateur ; gère l'authentification.
Gestionnaire	Hérite de Utilisateur	Gère produits, catégories, fournisseurs et mouvements.

Classe	Attributs principaux	Rôle
Superviseur	Hérite de Gestionnaire	Réalise les inventaires, consulte journal et statistiques.
Administrateur	Hérite de Utilisateur	Gère les accès et les utilisateurs.
Categorie	idCategorie, nom	Regroupe les produits.
Produit	idProduit, reference, nom, description, quantiteDisponible, seuilAlerte, prix	Produit suivi en stock ; indique si son stock est faible.
Fournisseur	idFournisseur, raisonSociale, codeFournisseur, personneContact, telephone, email, ville	Fournit des produits.
MouvementStock (abstraite)	idMouvement, dateMouvement, quantite	Partie commune aux entrées et sorties.
EntreStock	Hérite de MouvementStock	Augmente la quantité disponible.
SortieStock	Hérite de MouvementStock	Diminue la quantité disponible.
Alerte	idAlerte, dateAlerte, message, traitee	Signale un produit en stock faible.
Inventaire	idInventaire, dateInventaire	Opération de contrôle du stock.
LigneInventaire	idLigneInventaire, quantiteTheorique, quantiteReelle, ecart	Compare, pour un produit, quantité enregistrée et constatée.
JournalAudit	idAudit, dateHeure, action, module, typeObjet, idObjet, description	Trace les actions des utilisateurs.

3.3. Relations et règles de gestion

Tableau 4 : Relations entre les classes

Relation	Multiplicités	Signification
Categorie – Produit	1 – 0..*	Un produit appartient à une seule catégorie.
Produit – MouvementStock	1 – 0..*	Un produit a plusieurs entrées et sorties.
Gestionnaire – MouvementStock	1 – 0..*	Un gestionnaire enregistre plusieurs mouvements.
Gestionnaire – Fournisseur	1 – 0..*	Un gestionnaire gère plusieurs fournisseurs.
Fournisseur – Produit	0..* – 1	Un produit peut être fourni par des fournisseurs.
Produit – Alerte	1 – 0..*	Un produit peut générer plusieurs alertes.
Superviseur – Inventaire	1 – 0..*	Un superviseur réalise plusieurs inventaires.
Inventaire – LigneInventaire	1 – 1..*	Agrégation : un inventaire contient au moins une ligne.
Produit – LigneInventaire	1 – 0..*	Un produit apparaît dans plusieurs inventaires.
Gestionnaire – JournalAudit	1 – 0..*	Les actions d’un gestionnaire sont tracées.

Trois héritages structurent le modèle : Gestionnaire et Administrateur héritent de Utilisateur, Superviseur hérite de Gestionnaire, EntreStock et SortieStock héritent de MouvementStock.

Règles de gestion :

- chaque entrée augmente la quantité disponible du produit ;

- chaque sortie diminue la quantité disponible du produit ;
- une alerte est créée quand la quantité atteint le seuil d'alerte ;
- les statistiques ne sont pas stockées : elles sont calculées à partir des produits et des mouvements.

4. Modèle logique de données relationnel

4.1. Règles de passage

Tableau 5 : Passage du diagramme de classes au MLDR

Élément UML	Règle appliquée	Résultat
Classes persistantes	Une table par classe, avec clé primaire.	8 tables
Héritage Utilisateur	Table unique + colonne de rôle.	role_utilisateur
Héritage MouvementStock	Table unique + colonne de type.	type_mouvement
Association 1 – 0..*	La clé du côté 1 devient clé étrangère.	Clés étrangères
Inventaire – LigneInventaire	Clé étrangère avec suppression en cascade.	ON DELETE CASCADE
Attribut dérivé ecart	Colonne calculée : réelle − théorique.	GENERATED ALWAYS … STORED
Méthodes appliquer() et EstEnStockFaible()	Mise à jour du stock et alertes dans la base.	Triggers et vue SQL

La classe JournalAudit n'est pas encore traduite en table : c'est une évolution prévue.

4.2. Schéma relationnel

Légende : clé primaire en gras ; clé étrangère précédée de #.

UTILISATEUR (**id_utilisateur**, nom, prenom, email, mot_de_passe, statut, role_utilisateur)

FOURNISSEUR (**id_fournisseur**, raison_sociale, code_fournisseur, personne_contact, telephone, email, ville, #id_gestionnaire)

CATEGORIE (**id_categorie**, nom, #id_gestionnaire)

PRODUIT (**id_produit**, reference, nom, description, quantite_disponible, seuil_alerte, prix, #id_categorie, #id_gestionnaire)

MOUVEMENT_STOCK (**id_mouvement**, date_mouvement, quantite, type_mouvement, #id_produit, #id_gestionnaire)

INVENTAIRE (**id_inventaire**, date_inventaire, #id_superviseur)

LIGNE_INVENTAIRE (**id_ligne_inventaire**, quantite_theorique, quantite_reelle, ecart, #id_inventaire, #id_produit)

ALERTE (**id_alerte**, date_alerte, message, traitee, #id_produit)

5. Implémentation de la base de données

Le script SQL (Annexe A) est testé sur MariaDB 10.11 et compatible MySQL 8.0+. Il crée la base, les 8 tables, les contraintes, 10 index, une vue, 3 triggers et un jeu de données de test.

Traitements automatiques :

- une entrée ou une sortie met à jour la quantité disponible ;
- une sortie supérieure au stock est refusée (« Stock insuffisant ») ;
- une alerte est créée quand le stock passe sous le seuil (une seule alerte non traitée par produit) ;
- seul un utilisateur de rôle SUPERVISEUR peut réaliser un inventaire.

6. Réalisation de l'application

6.1. Environnement de développement

Tableau 6 : Technologies utilisées

Domaine	Technologie
Langage	TypeScript / JavaScript
Frameworks	ReactJS (interface) et NestJS (serveur)
Interface web	HTML, CSS, Tailwind CSS
Base de données	MySQL 8.0+ / MariaDB (InnoDB)
Outils	Visual Studio Code, Git

6.2. Interfaces de l'application

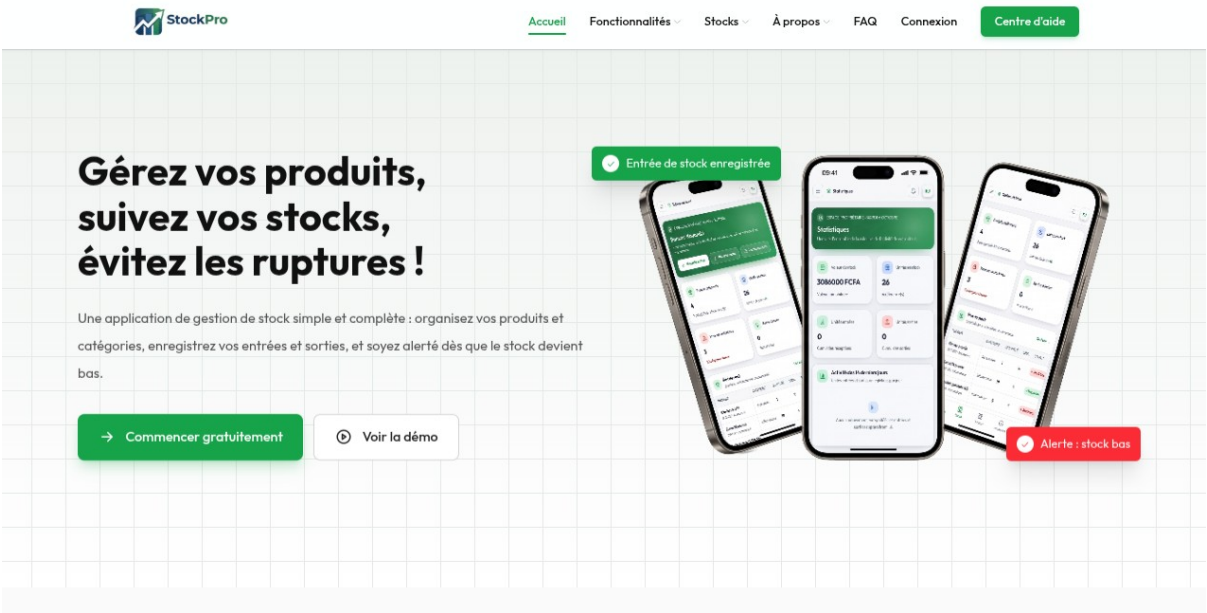


Figure 3 : Page d'accueil

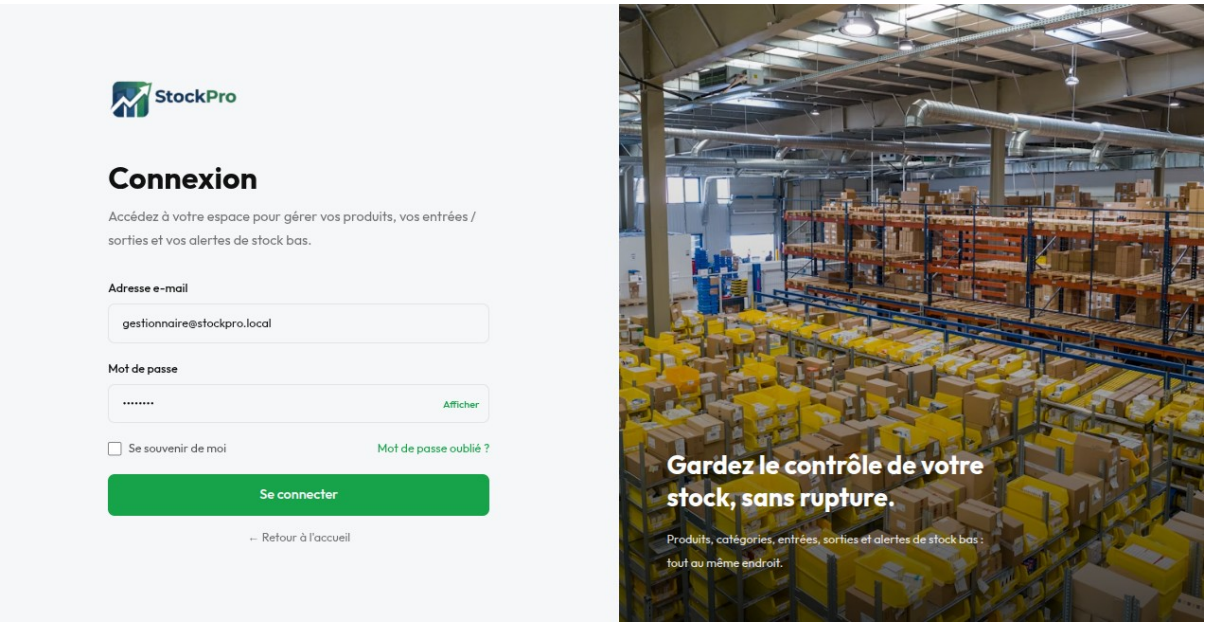


Figure 4 : Page de connexion

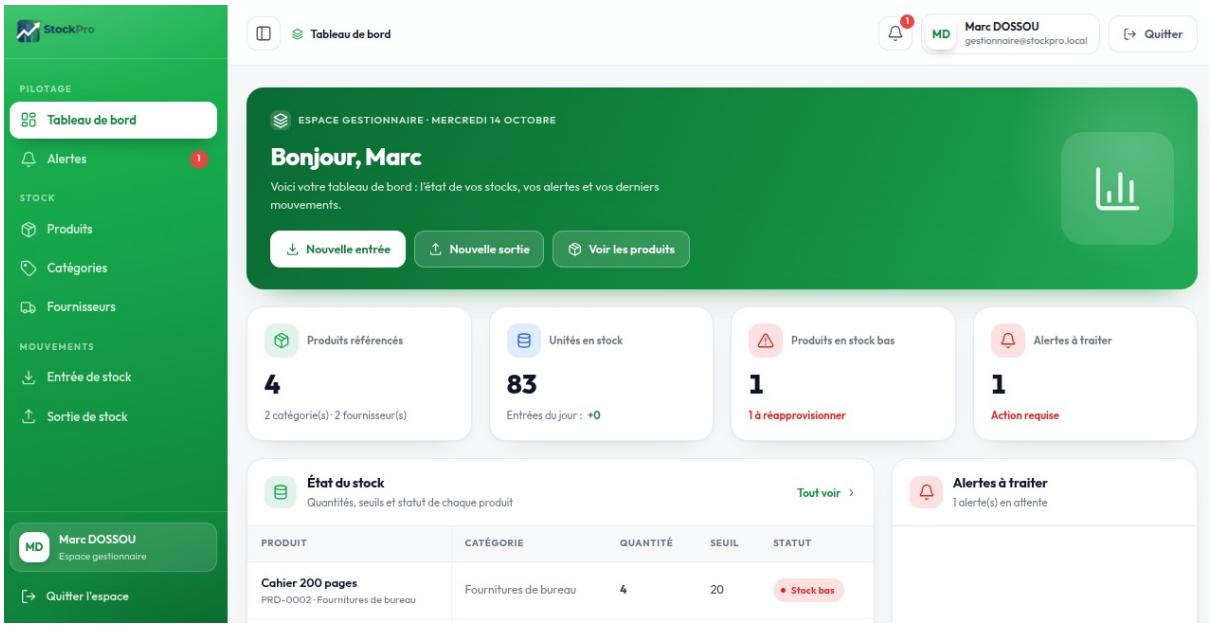


Figure 5 : Tableau de bord

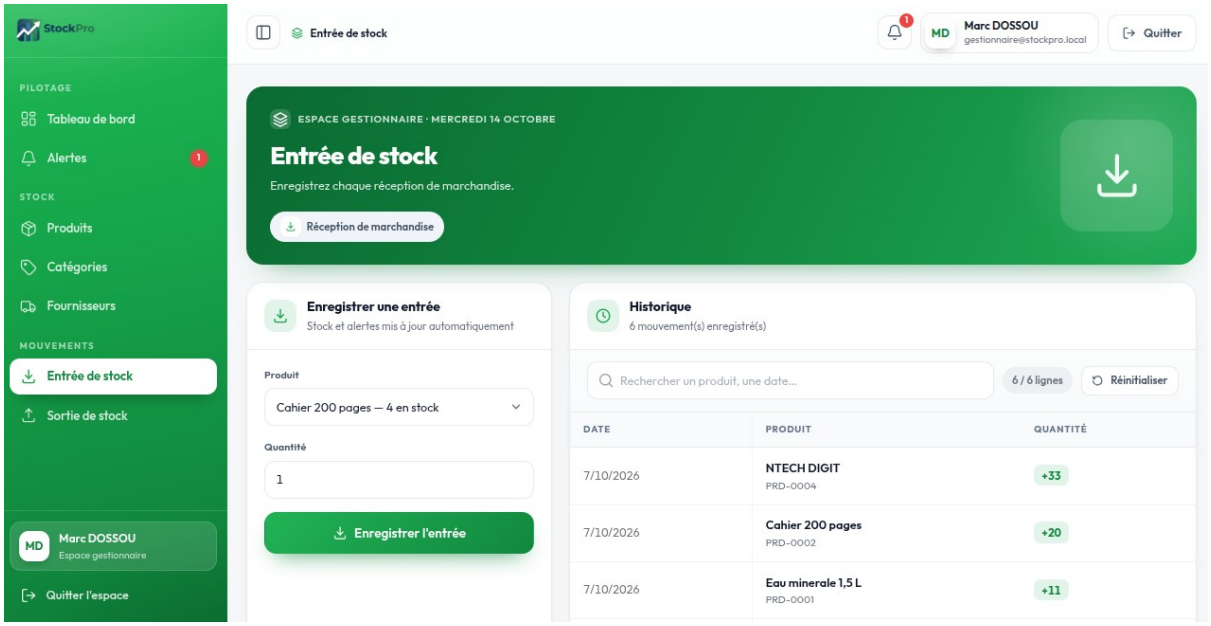


Figure 6 : Enregistrement d'une entrée

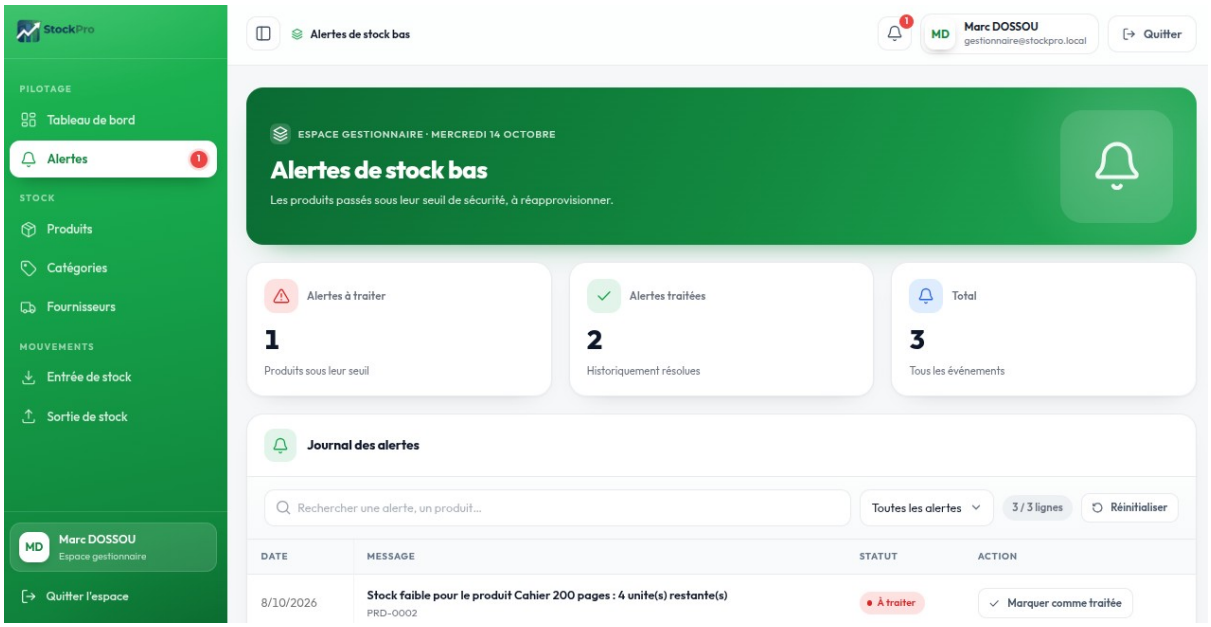


Figure 7 : Alertes de stock faible

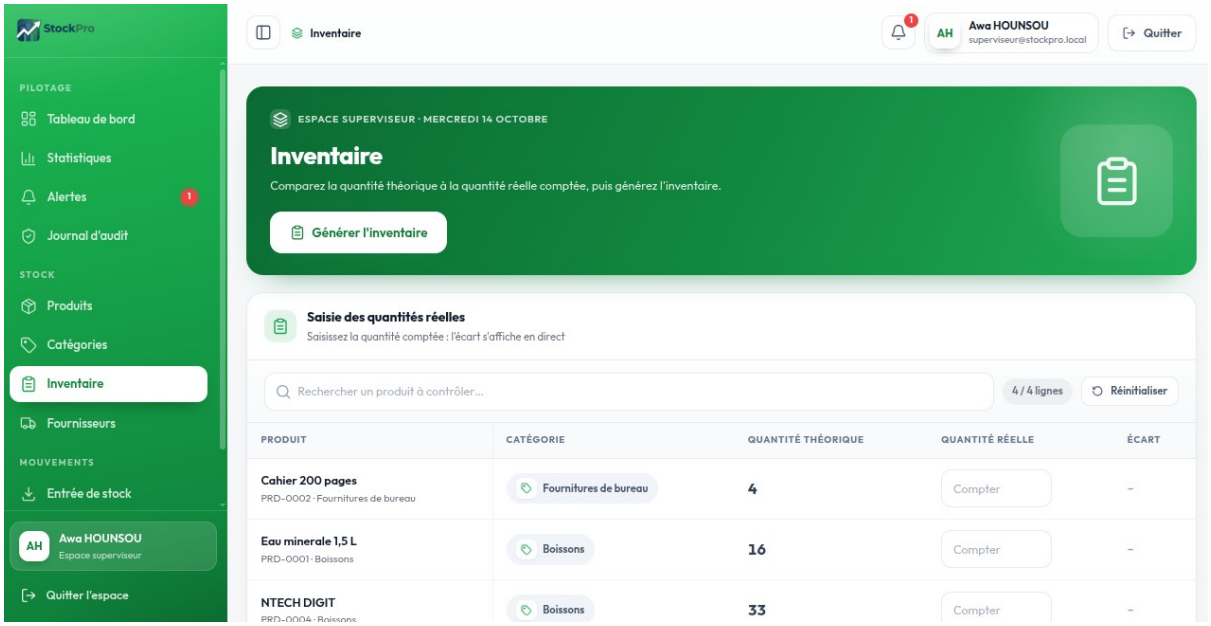


Figure 8 : Inventaire du stock (superviseur)

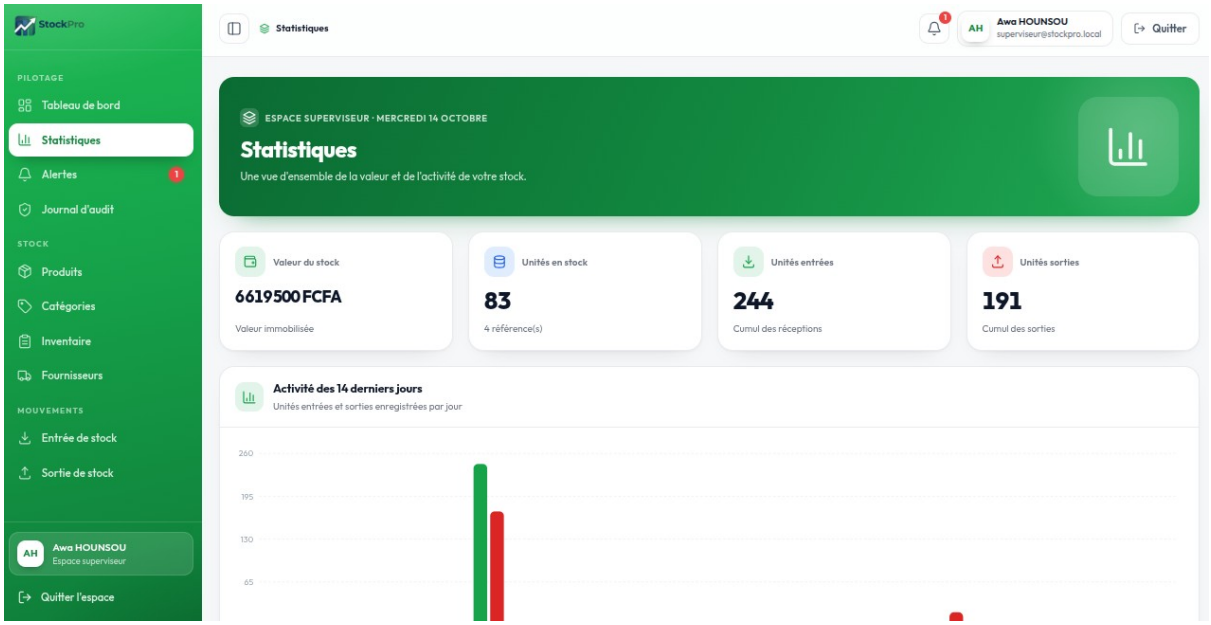


Figure 9 : Statistiques globales (superviseur)

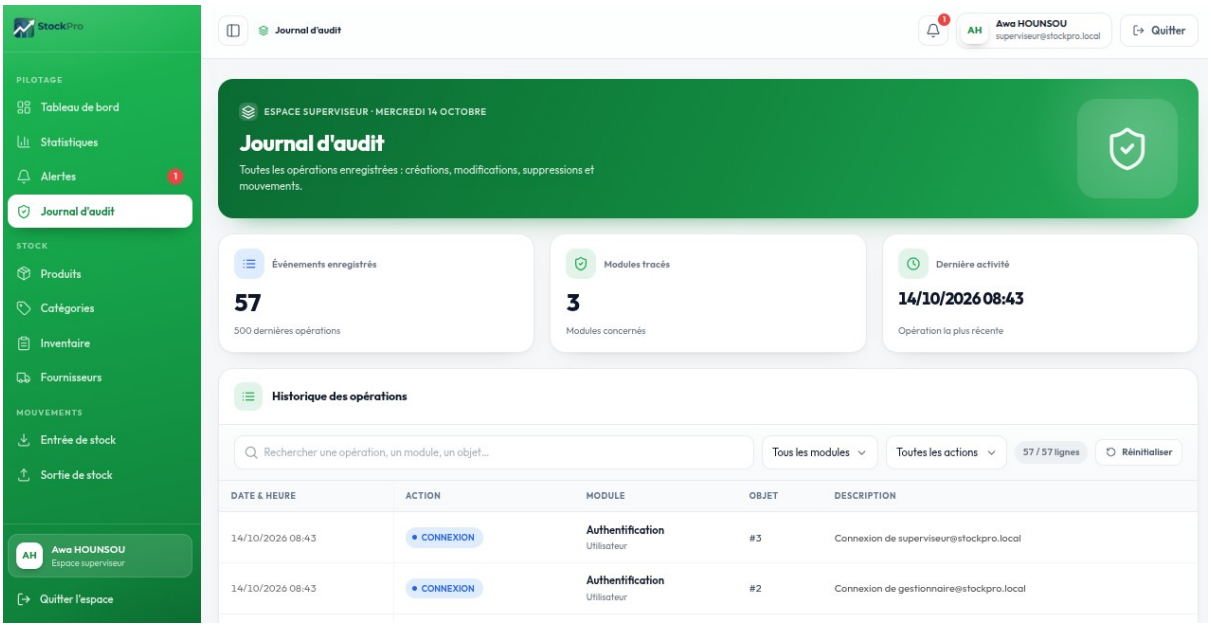


Figure 9 : Journal d'audit (superviseur)

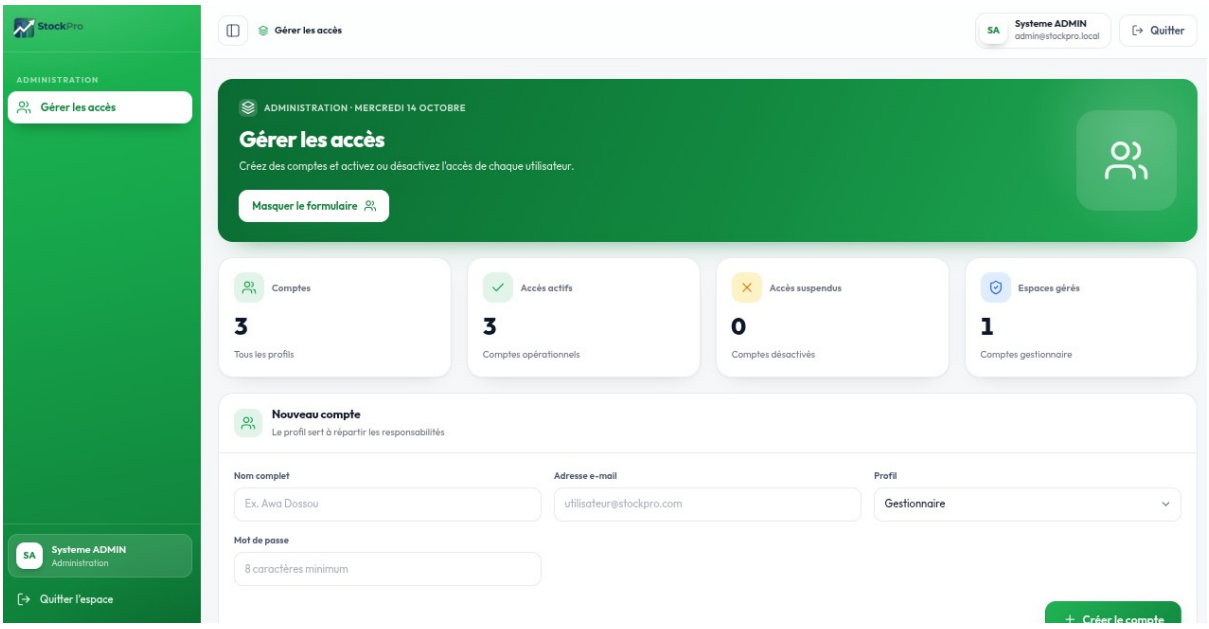


Figure 10 : Gestion des accès (administrateur)

7. Extraits de code

7.1. Authentification

Recherche de l’utilisateur :

const user = await this.prisma.utilisateur.findUnique({

where: { email }

});

Vérification du mot de passe :

if (!user || !verifyPassword(motDePasse, user.motDePasse)) {

throw new UnauthorizedException(

'E-mail ou mot de passe incorrect'

);

}

7.2. Gestion des produits

```
async addProduct(body, user) {
    const data = await this.produitData(body);
    const { quantite, ...champs } = data;

    const produit = await this.prisma.$transaction(async (tx) => {
        const cree = await tx.produit.create({
            data: {
                ...champs,
                quantite: 0,
                idGestionnaire: user.id
            },
        });

        if (quantite > 0) {
            await tx.mouvementStock.create({
                data: {
                    type: 'ENTREE',
                    quantite,
                    idProduit: cree.id,
                    idGestionnaire: user.id
                },
            });
        }

        return tx.produit.findUniqueOrThrow({
            where: { id: cree.id },
            include: { categorie: true }
        });
    });

    await syncAlerts(this.prisma);

    await this.audit.enregistrer({
        action: 'CREATION',
        module: 'Catalogue',
        typeObjet: 'Produit',
        idObjet: produit.id,
```

```
        description: `Produit ${produit.reference} (${produit.nom}) créé`,
    });

    return produitJson(produit);
}async addProduct(body, user) {
  const data = await this.produitData(body);
  const { quantite, ...champs } = data;

  const produit = await this.prisma.$transaction(async (tx) => {
    const cree = await tx.produit.create({
      data: {
        ...champs,
        quantite: 0,
        idGestionnaire: user.id,
      },
    });

    if (quantite > 0) {
      await tx.mouvementStock.create({
        data: {
          type: 'ENTREE',
          quantite,
          idProduit: cree.id,
          idGestionnaire: user.id,
        },
      });
    }

    return tx.produit.findUniqueOrThrow({
      where: { id: cree.id },
      include: { categorie: true },
    });
  });

  await syncAlerts(this.prisma);

  await this.audit.enregistrer({
    action: 'CREATION',
    module: 'Catalogue',
    typeObjet: 'Produit',
    idObjet: produit.id,
    description: `Produit ${produit.reference} (${produit.nom}) créé`,
  });

  return produitJson(produit);
}
```

Conclusion générale

Ce projet a abouti à StockPro, une application qui centralise les produits et leurs catégories, enregistre les entrées et les sorties et signale les stocks faibles. L’analyse a identifié trois acteurs (gestionnaire, superviseur, administrateur) ; la conception a produit un diagramme de classes traduit en un modèle relationnel de huit tables, où contraintes, index, vue et triggers automatisent la mise à jour du stock et les alertes.

La solution valide les trois hypothèses : suivi informatisé (H1), quantités exactes grâce aux mouvements (H2) et repérage rapide des stocks faibles (H3). Comme évolutions, nous envisageons d’ajouter la table du journal des activités, de relier les fournisseurs aux produits dans la base et d’enrichir les statistiques.